

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

**«Реализация метода Гаусса для действительной
квадратной матрицы с выбором ведущего элемента
для решения системы линейно алгебраических
уравнений.»**

Выполнил:

студент группы 3825Б1ПМиИ1
Огнев Роман Андреевич

Проверила:

Бусько П.В.

Нижний Новгород
2026

Содержание

Введение	3
Постановка задачи.....	4
Руководство пользователя	5
Описание программной реализации	5
Результаты экспериментов	8
Заключение.....	10
Литература	11
Приложение	12

Введение

В современном мире вычислительном математики решение системы линейно алгебраических уравнений (далее по тексту СЛАУ) является одной из фундаментальных задач. Данную систему можно представить в матричной форме: $Ax=b$, где A - матрица коэффициентов, b – вектор правой части, x – искомый неизвестный вектор.

В рамках данной лабораторной работы необходимо создать рабочую программу для решения СЛАУ с помощью метода Гаусса с выбором ведущего элемента, используя ООП. Программа будет находить значения неизвестных и выводить ответы на экран. По итогу необходимо предоставить рабочий код для решения СЛАУ. В коде будет построена иерархия объектов: «вектор», «матрица», «СЛАУ». Данный подход позволяет воспользоваться всеми преимуществами Объектно - Ориентированно Программирования, в нашем случае – наследование классов.



Таблица 1 Иерархия классов

Постановка задачи

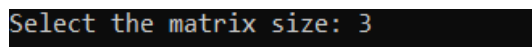
Цель: реализовать метод Гаусса с выбором ведущего элемента для действительной квадратной матрицы, используя шаблоны классов C++.

Задачи:

1. Разработать шаблонный класс вектор с перегрузкой арифметических операций.
2. Реализовать класс квадратной матрицы как шаблон класса вектор от вектора.
3. Создать класс СЛАУ – наследник класса матрицы – с методом Гаусса, принимающий правую часть уравнения в виде вектора и возвращающий вектор решений.
4. Реализовать дополнительную функцию для обмена строк(swapRows) для выбора ведущего элемента.
5. Реализовать защиту от вырожденной матрицы и от некорректного ввода пользователем.
6. Провести эксперименты, чтобы проверить правильность работы кода.

Руководство пользователя

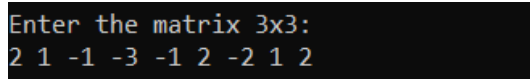
После запуска программы пользователю предлагается ввести размер желаемой матрицы, введя соответствующее целое число. Программа принимает любое натуральное число.



```
Select the matrix size: 3
```

Рисунок 1 Пример работы с программой. Ввод размера матрицы.

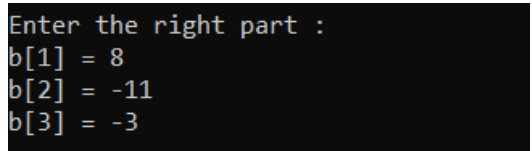
Далее пользователя просят ввести последовательно элементы матрицы.



```
Enter the matrix 3x3:  
2 1 -1 -3 -1 2 -2 1 2
```

Рисунок 2 Пример работы с программой. Последовательный ввод элементов матрицы.

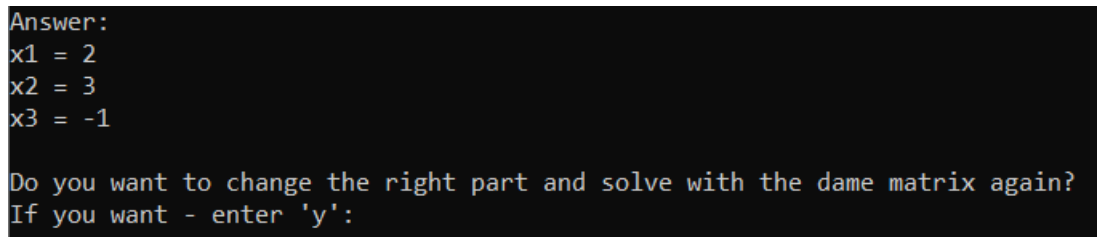
После пользователя просят ввести значения правой части.



```
Enter the right part :  
b[1] = 8  
b[2] = -11  
b[3] = -3
```

Рисунок 3 Пример работы с программой. Ввод правой части матрицы.

По итогу пользователь получает ответ. Далее программа предлагает пользователю продолжить вычисления СЛАУ, сохранив матрицу и введя с клавиатуры новую правую часть. Если пользователь введет «у», то программа попросит ввести правую часть заново. Введя любое другое значение, программа завершит работу.



```
Answer:  
x1 = 2  
x2 = 3  
x3 = -1  
  
Do you want to change the right part and solve with the same matrix again?  
If you want - enter 'y':
```

Рисунок 4 Пример работы с программой. Вывод ответа, предложение пользователю о новом вычислении СЛАУ с исходной матрицей.

При вводе нечислового значения программа выдает предупреждение и просит ввести данные заново. Если после преобразований на главной диагонали будет элемент равный нулю, то программа проверит значение правой части и выдаст сообщение о том, что система имеет множество решений, либо не имеет решения вовсе. В остальных случаях программа будет выводить поле с ответом. Разбор случаев см. в разделе [«Результаты экспериментов»](#).

Описание программной реализации

Мой проект представляет собой единый файл на языке C++ и состоит из 3 шаблонных классов, связанные отношением наследования.

Класс `My_vector<T>`. Базовый шаблонный класс, реализующий динамический массив произвольного типа. Хранит указатель на данные и размер массива:

1. `My_vector()` – Конструктор по умолчанию. Инициализирует пустой вектор.
2. `My_vector(size_t size, T value = T())` – Конструктор с параметром. Выделяет память и заполняет вектор заданным значением.
3. `My_vector(const My_vector& other)` – Конструктор копирования.
4. `My_vector operator=(const My_vector& other)` – Оператор присваивания. Освобождает старую память и копирует данные другого вектора.
5. `My_vector operator*(T scalar)` – Оператор умножения элементов вектора на скалярное значение.
6. `My_vector operator-(const My_vector& other)` – Оператор поэлементного вычитания двух векторов.
7. `T& operator[](size_t index)` – Оператор доступа к элементу по индексу. Есть две версии: обычная – для чтения и записи; `const` – только для чтения.
8. `~My_vector()` – Деструктор, освобождает динамическую выделенную память.

Класс `matrix<T>`. Шаблонный класс, наследующий `vector<vector<T>>`. Создает квадратную матрицу как вектор строк, каждая из которых является вектором элементов:

1. `matrix(size_t n) : n(n), My_vector<My_vector<T>>(n, My_vector<T>(n, T()))` – Конструктор, инициализирующий матрицу $n \times n$ нулями через вызов конструктора `My_vector<T>`.
2. `void swapRows(int row1, int row2)` – Функция замены двух строк местами с помощью временной переменной.
3. `void input()` – Функция последовательного ввода элементов матрицы с клавиатуры.

Класс `SLAY<T>`. Класс системы линейных алгебраических уравнений, наследованный от `matrix<T>`. В нём содержится основная вычислительная логика – Метод Гаусса:

1. `SLAY(size_t n) : matrix<T>(n)` – Конструктор, принимающий размер n и инициализирующий класс `matrix<T>`.

2. `My_vector<T> gauss(My_vector<T> b)` – Принимает вектор правой части и возвращает вектор решений. Реализует метод Гаусса с выбором ведущего элемента по столбцу. Всего есть два этапа:

- Прямой ход, который приводит матрицу к верхнетреугольному виду, то есть под главной диагональю нули. На каждом шаге среди строк ищется максимальное по модулю число в текущем столбце, оно становится ведущим элементом, после чего происходит замена строк. Потом находится множитель и строчка с ведущим элементом вычитается из всех нижестоящих строк.
- Обратный ход, нахождение значений неизвестных, начиная с последнего уравнения и поднимаясь вверх.

Перед делением выполняется проверка элемента на близость к нулю:

```
if (abs((*this)[i][i]) < 1e-10) {  
    T bi = b[i];  
    if (abs(bi) < 1e-10)  
}
```

Если на главной диагонали есть 0, то программа перейдет в следующий блок проверки. Программа проверит значений правой части. В зависимости от результата выдаст одно из двух сообщений. Подробнее см. в разделе [«Результаты экспериментов»](#).

3. `printSlay(My_vector<T> b)` – Выводит расширенную матрицу $[A|b]$ в форматированном виде с использованием `setw(8)` для выравнивания столбцов (`setw(n)` – манипулятор из библиотеки C++, который позволяет устанавливать ширину поля для последующих операций ввода-вывода, где n – количество вводимых/выводимых элементов).

Результаты экспериментов

Мною было проведено исследование работы алгоритма при разных условиях. Основной целью стало проверить, как работает программа при разных условиях ввода:

I. Проверка работоспособности, без вывода ошибок:

```
Select the matrix size: 2

Enter the matrix 2x2:
3 1
1 2

Enter the right part :
b[1] = 3
b[2] = 7

Start matrix:
      3      1 |      3
      1      2 |      7

Answer:
x1 = -0.2
x2 = 3.6
```

Рисунок 5 Матрица 2x2.

```
Select the matrix size: 3

Enter the matrix 3x3:
2 1 -1 -3 -1 2 -2 1 2

Enter the right part :
b[1] = 8
b[2] = -11
b[3] = -3

Start matrix:
      2      1     -1 |      8
     -3     -1      2 |     -11
     -2      1      2 |      -3

Answer:
x1 = 2
x2 = 3
x3 = -1
```

Рисунок 6 Матрица 3x3.

II. Проверка программы на вырожденную матрицу:

```
Select the matrix size: 3

Enter the matrix 3x3:
1 2 0
3 4 0
5 6 0

Enter the right part :
b[1] = 1
b[2] = 1
b[3] = 1

Start matrix:
      1      2      0 |      1
      3      4      0 |      1
      5      6      0 |      1

Infinitely many solutions.
System does haven't a unique solution.
```

Рисунок 7 Проверка матриц на вырожденность. 3 столбец нулевой.

```
Select the matrix size: 2

Enter the matrix 2x2:
1 2
2 4

Enter the right part :
b[1] = 1
b[2] = 4

Start matrix:
      1      2 |      1
      2      4 |      4

There are no solutions.
System does haven't a unique solution.
```

Рисунок 8 Проверка матрицы на вырожденность. Второй столбец - удвоенный первый.

Программа оснащена проверкой матрицы на нулевые элементы по диагонали, т. к. метод Гаусса включает деление правой части системы на левую, поэтому необходимо избежать деления на 0. В результатах эксперимента видно, что программа успешно распознает 0 на главной диагонали и выводит разные сообщения в зависимости от того, чему равна правая

часть в строчке проверяемого элемента. Если $x[i]=0$ и $\text{abs}(b[i])>0$ – то система не имеет решения. Если $x[i]=0$ и $b[i]=0$ – то система имеет множество решений.

III. Проверка сообщений об ошибочном вводе значений:

```
Select the matrix size: abs
Error. Please enter a positive number: 2

Enter the matrix 2x2:
10 6
3 5

Enter the right part :
b[1] = 1
b[2] = 2

Start matrix:
    10    6 |    1
    3     5 |    2

Answer:
x1 = -0.21875
x2 = 0.53125
```

Рисунок 9 Проверка на ввод размера матрицы.

```
Select the matrix size: 2

Enter the matrix 2x2:
3 1
abs 3
Incorrect input matrix! Enter numbers: 4 2

Enter the right part :
b[1] = 1
b[2] = 4

Start matrix:
    3    1 |    1
    4    2 |    4

Answer:
x1 = -1
x2 = 4
```

Рисунок 10 Проверка на ввод элементов матрицы.

```
Select the matrix size: 2

Enter the matrix 2x2:
3 1 1 4

Enter the right part :
b[1] = 1
b[2] = abs
Incorrect input right part! Enter numbers: 4

Start matrix:
    3    1 |    1
    1    4 |    4

Answer:
x0 = 0
x2 = 1
```

Рисунок 11 Проверка на ввод правой части системы

Как видно из результатов экспериментов. Программа чётко распознает не числовой символ («abs») и просит пользователя ввести значения заново. После повторного правильного ввода, программа выдаст результат.

Благодаря экспериментам можно сделать вывод, что код работает корректно. Программа успешно находит и выводит значение неизвестных из введенных параметров. Все защитные механизмы отработали штатно. Исследование вырожденных матриц подтвердило, что определение существования решений работает исправно.

Заключение

В ходе лабораторной работы, был реализован рабочий код на языке «C++» для вычисления значений из СЛАУ методом Гаусса с выбором ведущего элемента.

Использование шаблонных классов позволило создать универсальное решение, благодаря которому можно использовать разные числовые типы (int, double, float). Создание классов наследования позволило обеспечить логическое разделение функциональности. Например, операции над векторами реализованы один раз и переиспользуются во всех других классах наследования.

Выбор ведущего элемента по столбцу позволяет избежать деления на ноль. Тестирование подтвердило корректность реализации на матрицах размером 2×2 и 3×3 , а также корректную обработку вырожденных матриц и ошибочного ввода пользователя.

Литература

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.
2. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. / М.: Издательский дом «Вильямс», 2011.
3. Язык программирования С | Керниган Брайан У., Ритчи Деннис М. - Издательский дом Вильямс, 2016.

Приложение

```
template <typename T>
class My_vector {
private:
    T* data;
    size_t size;
public:
    My_vector() {
        data = nullptr;
        size = 0;
    }
    My_vector(size_t size, T value = T()) {
        this->size = size;
        data = new T[size];
        for (int i = 0; i < size; i++) {
            data[i] = value;
        }
    }
    My_vector(const My_vector& other) {
        size = other.size;
        data = new T[size];
        for (int i = 0; i < size; i++) {
            data[i] = other.data[i];
        }
    }
    My_vector operator=(const My_vector& other) {
        if (this != &other) {
            delete[] data;
            size = other.size;
            data = new T[size];
            for (int i = 0; i < size; i++) {
                data[i] = other.data[i];
            }
        }
        return *this;
    }
    My_vector operator*(T scalar) {
        My_vector result(size, T());
        for (int i = 0; i < size; i++) {
            result[i] = data[i] * scalar;
        }
        return result;
    }
    My_vector operator-(const My_vector& other) {
        My_vector result(size);
        for (int i = 0; i < size; i++) {
            result[i] = data[i] - other.data[i];
        }
        return result;
    }
    T& operator[](size_t index) {
        return data[index];
    }
    const T& operator[](size_t index) const {
        return data[index];
    }
    ~My_vector() {
        delete[] data;
    }
};
```

```

template<typename T>
class matrix:public My_vector<My_vector<T>> {
protected:
    size_t n;
public:
    matrix(size_t n) : n(n), My_vector<My_vector<T>>(n, My_vector<T>(n, T())) {};
    void swapRows(int row1, int row2) {
        My_vector<T> temp = (*this)[row1];
        (*this)[row1] = (*this)[row2];
        (*this)[row2] = temp;
    }
    void input() {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                while (!(cin >> (*this)[i][j])) {
                    cout << "Incorrect input matrix! Enter numbers: ";
                    cin.clear();
                    cin.ignore(1000, '\n');
                }
            }
        }
    }
};

```

Приложение 2 Класс Matrix

```

template<typename T>
class SLAY : public matrix<T> {
public:
    bool singular = false;
    SLAY(size_t n) : matrix<T>(n) {}
    My_vector<T> gauss(My_vector<T> b) {
        singular = false;
        size_t n = this->n;
        for (int i = 0; i < n; i++) {
            int maxRow = i;
            for (int k = i + 1; k < n; k++) {
                if (abs((*this)[k][i]) > abs((*this)[maxRow][i]))
                    maxRow = k;
            }
            this->swapRows(i, maxRow);
            T temp = b[i];
            b[i] = b[maxRow];
            b[maxRow] = temp;
            if (abs((*this)[i][i]) < 1e-10) {
                T bi = b[i];
                if (abs(bi) < 1e-10) cout << "\nInfinitely many solutions.";
                else cout << "\nThere are no solutions.";
                singular = true;
                return My_vector<T>(n, T());
            }
            for (int j = i + 1; j < n; j++) {
                T scalar = (*this)[j][i] / (*this)[i][i];
                (*this)[j] = (*this)[j] - (*this)[i] * scalar;
                b[j] = b[j] - b[i] * scalar;
            }
        }
        My_vector<T> x(n, T());
        for (int i = n - 1; i >= 0; i--) {
            x[i] = b[i];
            for (int j = i + 1; j < n; j++) {
                x[i] = x[i] - (*this)[i][j] * x[j];
            }
            x[i] = x[i] / (*this)[i][i];
        }
        return x;
    }
    void printSlay(My_vector<T> b) {
        size_t n = this->n;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                cout << setw(8) << (*this)[i][j];
            }
            cout << " |" << setw(8) << b[i] << endl;
        }
    }
};

```